

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине: «Основы машинного обучения»
Тема: «**Введение в нейронные сети: построение многослойного
перцептрона**»

Выполнил:
Студент 3 курса
Группы АС-65
Хвисюк К. Г.
Проверил:
Крощенко А. А.

Брест 2025

Цель работы: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации.

Общий план для всех вариантов:

1. Импорт библиотек и подготовка данных

- импортируйте torch, torch.nn, torch.optim, а также sklearn для загрузки данных и их предобработки;
- загрузите датасет, выполните стандартизацию (StandardScaler) и кодирование признаков;
- разделите данные на обучающую и тестовую выборки;
- преобразуйте данные (признаки и метки) в тензоры PyTorch: `torch.tensor(X_train, dtype=torch.float32)`.

2. Определение архитектуры нейронной сети

- создайте класс, наследуемый от `torch.nn.Module`;
- в методе `__init__` определите все слои, которые будете использовать (например, `nn.Linear`, `nn.ReLU`, `nn.Dropout`);
- в методе `forward` опишите последовательность применения слоев к входным данным.

3. Инициализация модели, функции потерь и оптимизатора

- создайте экземпляр вашей модели: `model = MLP()`;
- определите функцию потерь. Для бинарной классификации используйте `nn.BCEWithLogitsLoss`, для многоклассовой – `nn.CrossEntropyLoss`;
- определите оптимизатор: `optimizer = torch.optim.Adam(model.parameters(), lr=0.001)`.

4. Написание цикла обучения (Training Loop)

- запустите цикл на определенное количество эпох;
- внутри цикла:
 1. переведите модель в режим обучения: `model.train()`;
 2. сделайте предсказание (forward pass): `y_pred = model(X_train)`;
 3. рассчитайте потери (loss): `loss = criterion(y_pred, y_train)`;
 4. обнулите градиенты: `optimizer.zero_grad()`;
 5. выполните обратное распространение `loss.backward()`;
 6. сделайте шаг оптимизации: `optimizer.step()`.

5. Оценка модели (Evaluation)

- переведите модель в режим оценки: `model.eval()`;
- используйте `with torch.no_grad():`, чтобы отключить расчет градиентов;
- сделайте предсказания на тестовых данных;
- преобразуйте выходные данные (логиты) в предсказанные классы (например, с помощью `torch.argmax` или проверки порога `> 0`);
- рассчитайте метрики (accuracy, f1-score и т.д.), используя `sklearn.metrics`.

Вариант 9

Определение вида стекла

Glass Identification

Задача: классифицировать тип стекла (многоклассовая задача).

Архитектура:

- входной слой;
- два скрытых слоя по 10 нейронов в каждом (ReLU);
- выходной слой с необходимым количеством нейронов (Softmax).

Эксперимент: попробуйте обучить модель с одним скрытым слоем на 20 нейронов. Какая архитектура оказалась лучше?

Код программы:

```
import torch
import torch.nn as nn
import torch.optim as optim
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, f1_score, classification_report
import numpy as np

data = pd.read_csv("glass.csv")

X = data.drop("Type", axis=1).values
y = data["Type"].values

unique_classes = np.sort(np.unique(y))
class_to_index = {c: i for i, c in enumerate(unique_classes)}
y = np.array([class_to_index[c] for c in y])

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.25, random_state=16, stratify=y
)

X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

num_features = X_train.shape[1]
num_classes = len(unique_classes)

class MLP_two_hidden(nn.Module):
    def __init__(self, input_size, num_classes):
        super(MLP_two_hidden, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(input_size, 10),
            nn.ReLU(),
            nn.Linear(10, 10),
            nn.ReLU(),
            nn.Linear(10, num_classes)
        )
```

```

def forward(self, x):
    return self.net(x)

class MLP_one_hidden(nn.Module):
    def __init__(self, input_size, num_classes):
        super(MLP_one_hidden, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(input_size, 20),
            nn.ReLU(),
            nn.Linear(20, num_classes)
        )

    def forward(self, x):
        return self.net(x)

model1 = MLP_two_hidden(num_features, num_classes)
model2 = MLP_one_hidden(num_features, num_classes)

criterion = nn.CrossEntropyLoss()
optimizer1 = optim.Adam(model1.parameters(), lr=0.001)
optimizer2 = optim.Adam(model2.parameters(), lr=0.001)

def train_model(model, optimizer, X_train, y_train, epochs=100):
    for epoch in range(epochs):
        model.train()
        y_pred = model(X_train)
        loss = criterion(y_pred, y_train)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    return model

model1 = train_model(model1, optimizer1, X_train, y_train)
model2 = train_model(model2, optimizer2, X_train, y_train)

def evaluate_model(model, X_test, y_test):
    model.eval()
    with torch.no_grad():
        outputs = model(X_test)
        _, preds = torch.max(outputs, 1)
    acc = accuracy_score(y_test, preds)
    f1 = f1_score(y_test, preds, average="weighted")
    return acc, f1, preds

acc1, f1_1, preds1 = evaluate_model(model1, X_test, y_test)
acc2, f1_2, preds2 = evaluate_model(model2, X_test, y_test)

print("\nРезультаты")
print(f"2 скрытых слоя по 10 нейронов: Accuracy = {acc1:.3f}, F1 = {f1_1:.3f}")
print(f"1 скрытый слой на 20 нейронов: Accuracy = {acc2:.3f}, F1 = {f1_2:.3f}")

print("\nОценка моделей")
print(classification_report(y_test, preds1))
print(classification_report(y_test, preds2))

```

Результаты

```

2 скрытых слоя по 10 нейронов: Accuracy = 0.500, F1 = 0.443
1 скрытый слой на 20 нейронов: Accuracy = 0.463, F1 = 0.416

```

2 скрытых по 10:

	precision	recall	f1-score	support
0	0.38	0.28	0.32	18
1	0.46	0.84	0.59	19
2	0.00	0.00	0.00	4
3	0.00	0.00	0.00	3
4	0.00	0.00	0.00	2
5	1.00	0.75	0.86	8
accuracy			0.50	54
macro avg	0.31	0.31	0.30	54
weighted avg	0.44	0.50	0.44	54

1 скрытый на 20:

	precision	recall	f1-score	support
0	0.40	0.33	0.36	18
1	0.40	0.63	0.49	19
2	0.00	0.00	0.00	4
3	0.00	0.00	0.00	3
4	0.00	0.00	0.00	2
5	0.78	0.88	0.82	8
accuracy			0.46	54
macro avg	0.26	0.31	0.28	54
weighted avg	0.39	0.46	0.42	54

Вывод: построили, обучили и оценили многослойный перцептрон (MLP) для решения задачи классификации.